

Методы оптимизации в машинном обучении

Лекция № 1

Ю.Т. Каганов к.т.н., доцент

Кафедра ИУ-5

МГТУ им. Н.Э. Баумана

План

1. Обучение. Понятие оптимальности.
 2. Математическое программирование. Основные понятия.
 3. Выпуклые функции.
 4. Дифференцирование функций и поиск экстремума.
 5. Методы оптимизации в машинном обучении.
 6. Градиентные методы решения задач в МО.
 - Стохастический градиентный спуск
 - Проблемы обусловленности и овражности
 - Выбор learning rate
 - Целевые функции в МО
 - Квазиньютоновские методы (BFGS)
- ## 2. Адаптивные методы
- Метод Поляка (Momentum)
 - Nesterov Accelerated Gradient
 - AdaGrad, RMSProp, Adam, NAdam

Обучение

Обучение — это процесс приобретения новых знаний, навыков, умений или поведения в результате изучения, опыта или практики.

Обучение как философская категория

В философии обучение рассматривается не просто как процесс усвоения знаний, а как фундаментальный феномен человеческого существования, связанный с познанием, становлением личности и взаимодействием с миром.

Гносеологический аспект (теория познания)

1. **Рационализм** (Декарт, Кант) — обучение как процесс логического осмысления и открытия истин разумом.
2. **Эмпиризм** (Локк, Юм) — знание возникает из опыта, обучение основано на чувственном восприятии.
3. **Конструктивизм** (Пиаже, Выготский) — знание не "передаётся", а конструируется в процессе взаимодействия с миром.

Машинное обучение (Machine Learning, ML)

Машинное обучение (Machine Learning, ML) — это подраздел искусственного интеллекта (ИИ), изучающий методы, которые позволяют компьютерам *автоматически обучаться* на данных *без явного программирования*. В отличие от традиционного ПО, где правила жёстко заданы, ML-алгоритмы выявляют закономерности и *самостоятельно улучшают* свои прогнозы или решения.

◆ Обучение на данных

ML основано на **статистических закономерностях**: алгоритм анализирует входные данные (например, изображения, текст, числа) и находит скрытые взаимосвязи.

Чем больше и качественнее данные, тем лучше модель "учится".

◆ Адаптивность

Модель не просто запоминает примеры, а **обобщает** закономерности для работы с новыми, неизвестными данными.

Пример: распознавание рукописных цифр (даже если почерк новый).

Сущность машинного обучения — в **автоматическом выявлении закономерностей из данных** и способности к *самооптимизации*. Оно меняет парадигму программирования, заменяя "жёсткую логику" на адаптивные модели, но поднимает вопросы интерпретируемости и этики.

Понятие оптимальности

Понятие оптимальности предполагает наличие критериев оценки качества решения. При этом должно достигаться экстремальное значение этих критериев при удовлетворении некоторых ограничивающих условий.

С точки зрения математики задачи, поставленные таким образом, называются экстремальными. Это задачи, для которых ищется экстремум — т.е. минимум или максимум некоторой функции или функционала при заданных ограничениях.

Решение задачи состоит в поиске экстремума путем модификации параметров таким образом, чтобы целевая функция приближалась к экстремуму.

$$\begin{aligned} &extr (min, max) f(x, y, \theta) \\ &\theta_{i+1} = \theta_i + \Delta\theta_i \\ &f(x, y, \theta_{i+1}) > f(x, y, \theta_i) \end{aligned}$$

Математическое программирование:

ОСНОВНЫЕ ПОНЯТИЯ

Математическое программирование — это раздел математики, занимающийся поиском экстремумов (максимумов или минимумов) функций при заданных ограничениях. Оно широко применяется в экономике, технике, управлении и других областях для оптимизации решений.

Основные понятия

1.1. Оптимизационная задача

Любая задача математического программирования включает:

- **Целевую функцию** $f(x)$ — функцию, которую нужно максимизировать или минимизировать.
- **Переменные решения** $x = (x_1, x_2, \dots, x_n)$ — параметры, которые можно изменять.
- **Ограничения** — условия, которым должны удовлетворять переменные (равенства или неравенства).

Общий вид задачи:

$$\begin{aligned} &\max \text{ (или } \min) f(x), \\ &g_i(x) \leq 0, \quad i=1, \dots, m, \\ &h_j(x) = 0, \quad j=1, \dots, k. \end{aligned}$$

Математическое программирование:

ОСНОВНЫЕ ПОНЯТИЯ

В зависимости от вида целевой функции и ограничений выделяют:

1. Линейное программирование (ЛП)

1. Целевая функция и ограничения линейны.
2. Пример: задача оптимального распределения ресурсов.

2. Нелинейное программирование (НЛП)

1. Целевая функция или ограничения нелинейны.
2. Пример: задача минимизации квадратичной функции.

3. Целочисленное программирование

1. Переменные принимают только целые значения.
2. Пример: задача о рюкзаке или коммивояжёре.

4. Динамическое программирование

1. Оптимизация многошаговых процессов (метод Беллмана).

5. Выпуклое программирование

1. Целевая функция выпукла, ограничения задают выпуклое множество.

6. Стохастическое программирование

1. Учитывает случайные параметры в модели.

7. Неопределенное программирование

1. Использование теории нечетких множеств.

Математическое программирование:

ОСНОВНЫЕ ПОНЯТИЯ

Допустимое и оптимальное решение

- **Допустимое решение** — удовлетворяет всем ограничениям.
- **Оптимальное решение** — допустимое решение, дающее экстремум целевой функции.

Градиент и условия оптимальности

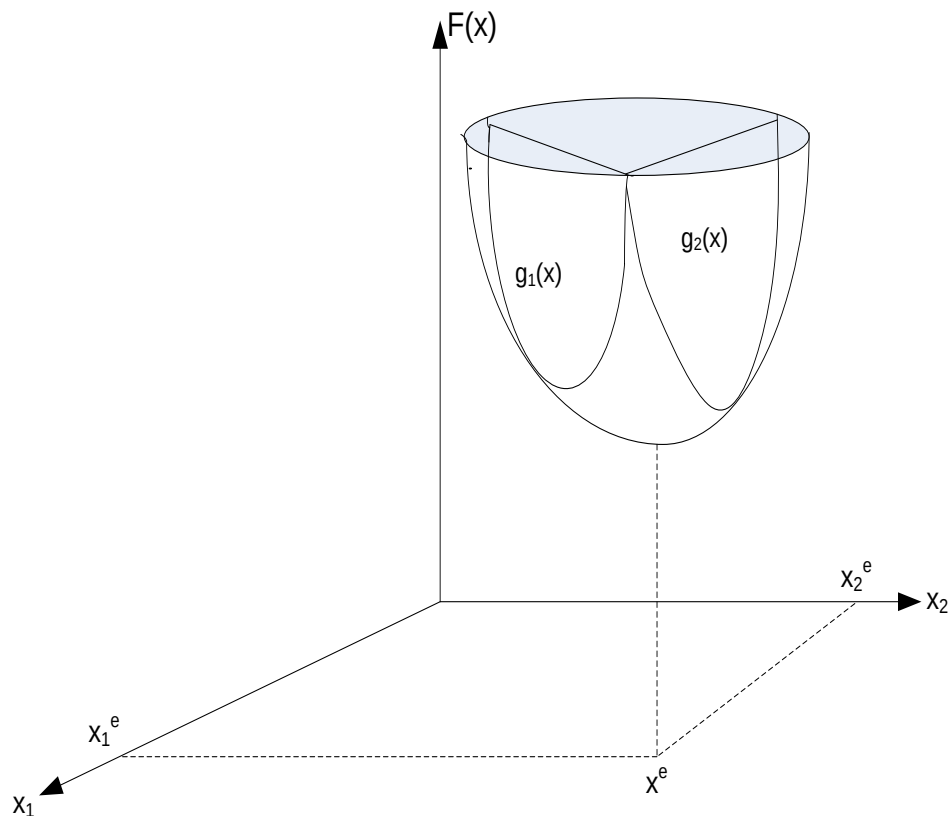
- **Градиент $\nabla f(x)$** — вектор частных производных, указывающий направление наискорейшего роста функции.
- **Условия Куна-Таккера** — обобщение метода множителей Лагранжа для задач с ограничениями.

Методы решения

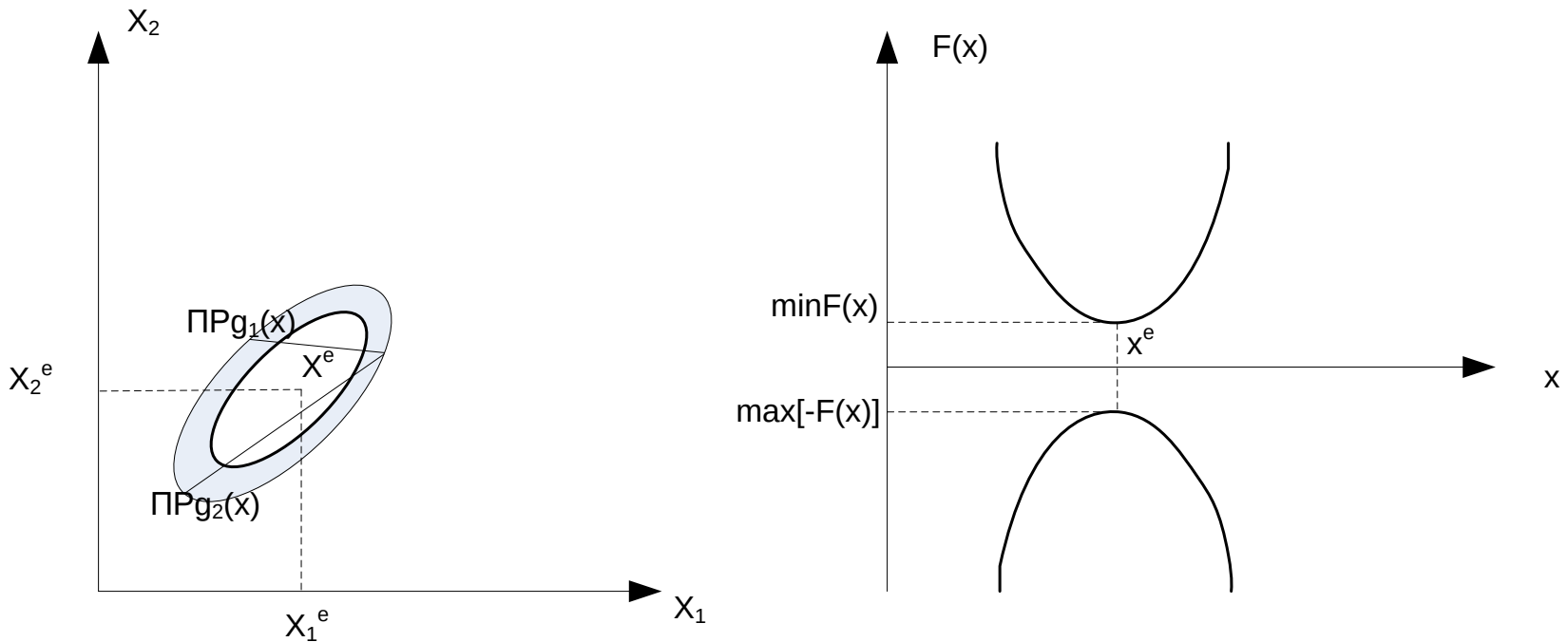
- **Симплекс-метод** (для линейных задач).
- **Метод Ньютона, градиентный спуск** (для нелинейных задач).
- **Метод ветвей и границ** (для целочисленных задач).
- **Динамическое программирование** (для многоэтапных задач).

Понятие оптимальности

- Решение задачи состоит в поиске **экстремума** целевой функции (**критерия оптимальности**) при заданных **ограничениях** и уравнениях состояния.



Понятие оптимальности

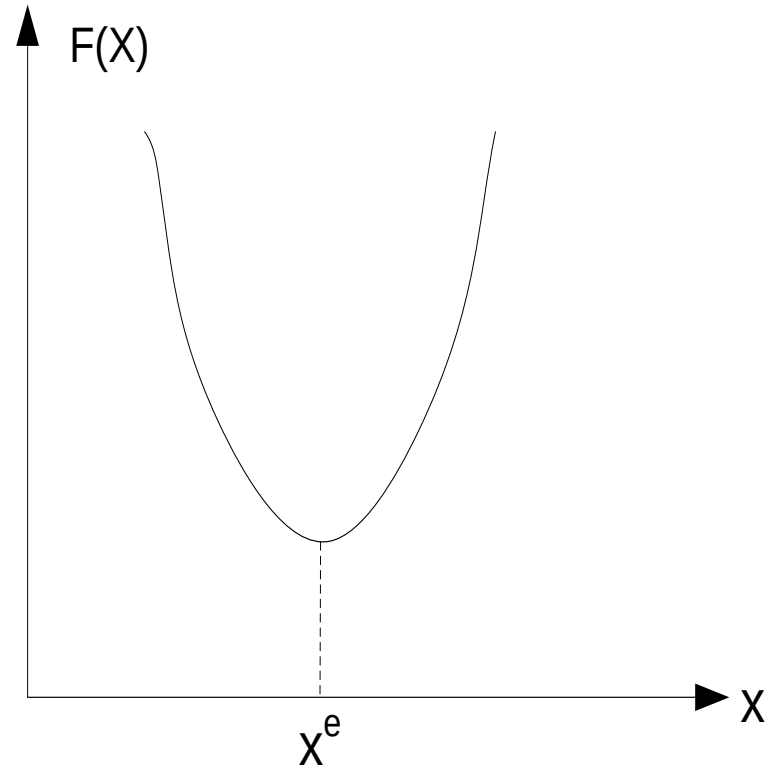


Задача *минимизации* функции (функционала) всегда может быть переформулирована как задача *максимизации* и наоборот.

Один экстремум (Унимодальная функция)

Виды задач:

- методы *одноэкстремальной* оптимизации (единственный экстремум — *унимодальная* функция);

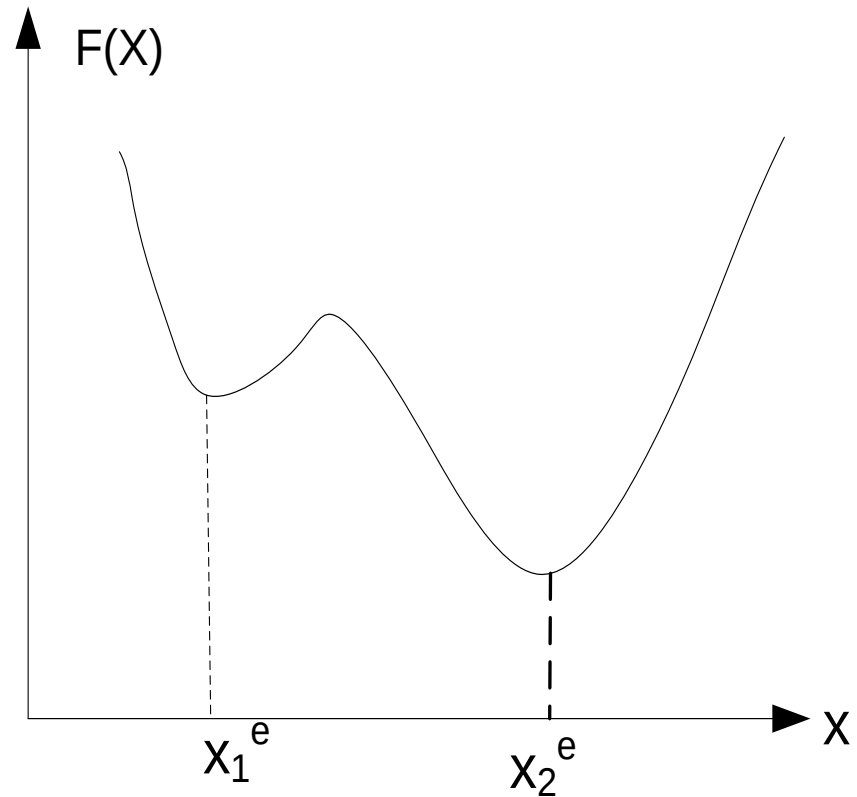


Много экстремумов (Полимодалая функция)

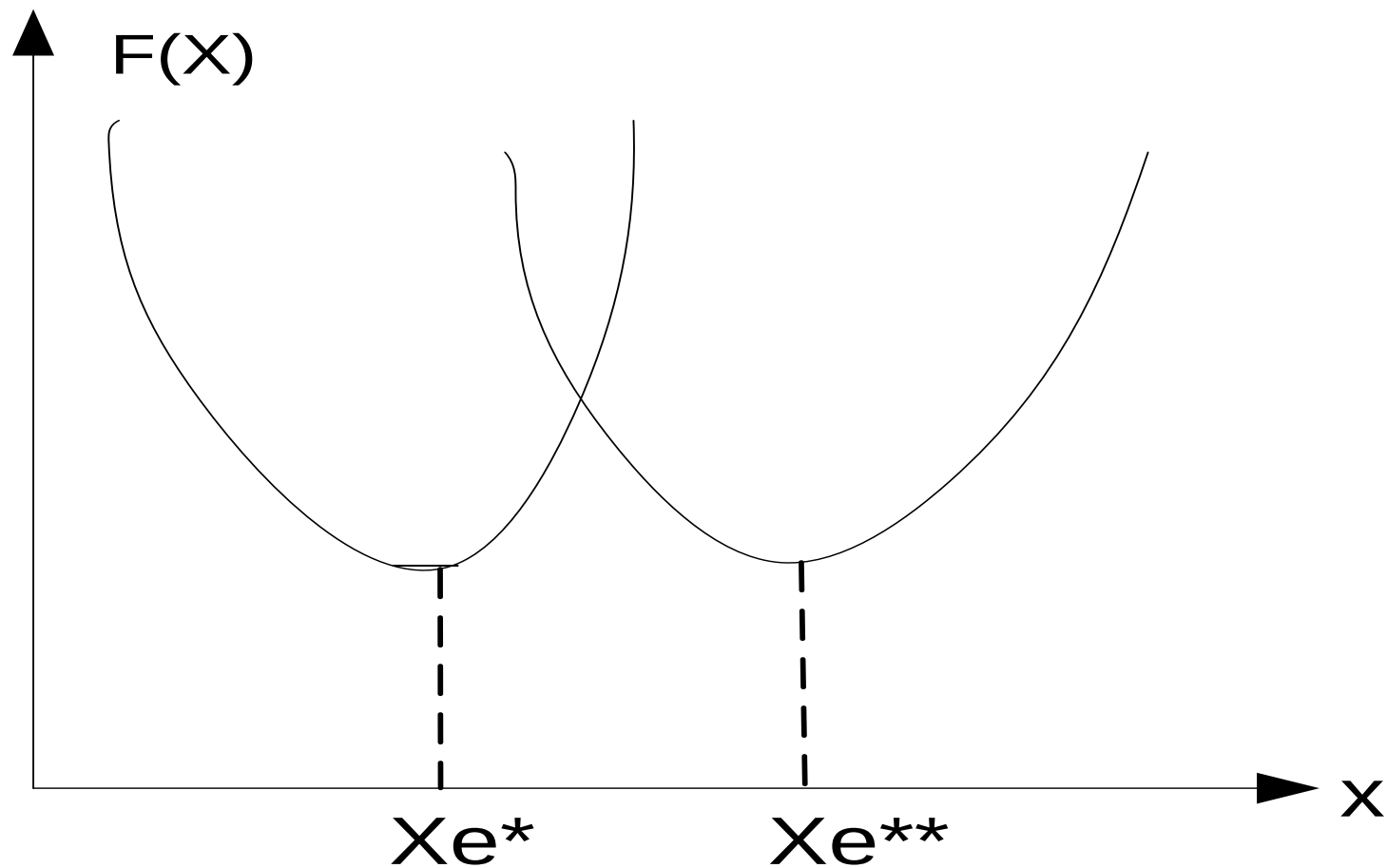
Виды задач:

методы

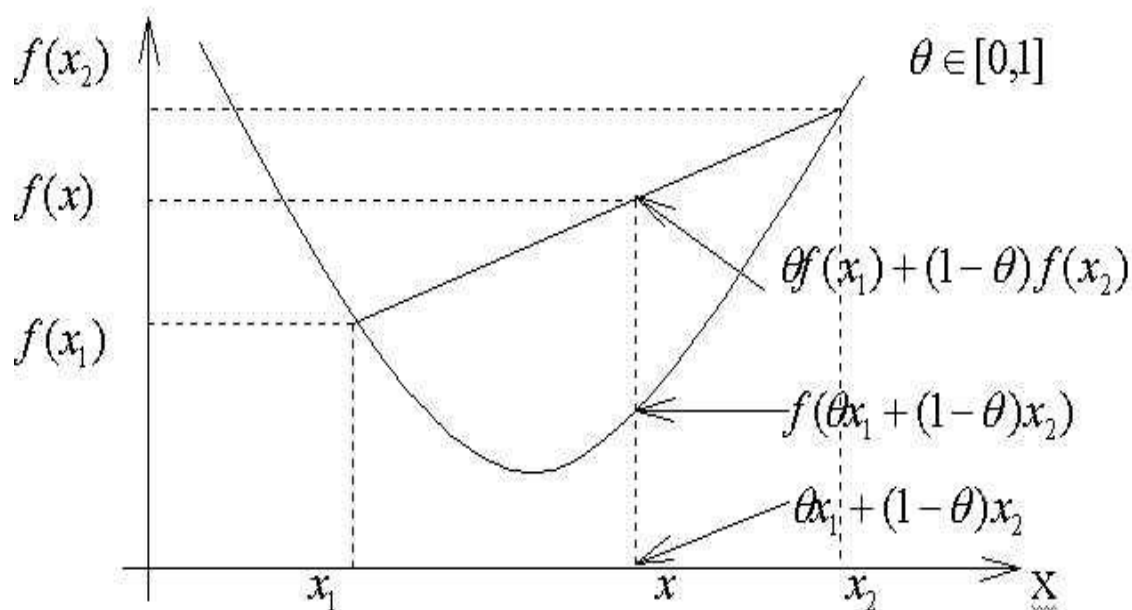
многоэкстремальной
оптимизации (много
экстремумов —
полимодалая
функция);



Многокритериальная оптимизация



Выпуклые функции



Выпуклые (вниз) функции.

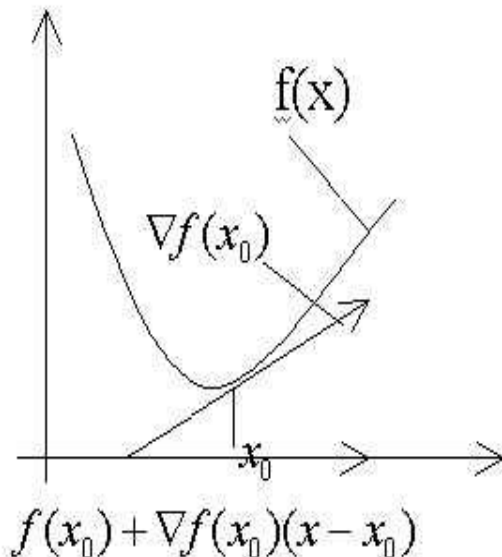
Выпуклой будем называть функцию, которая удовлетворяет следующему условию:

$$f(x) = \{y \in Y, y \in f(x) \mid (\forall x_1, x_2 \in X) \& (\forall \theta \in [0, 1]) \Rightarrow f[\theta x_1 + (1 - \theta)x_2] < \theta f(x_1) + (1 - \theta)f(x_2)\}$$

Дифференцирование функций.

Градиент функции.

Пусть $f: X \rightarrow Y; X \subset E^n, Y \subset E^1$ $f(x)$ – всюду гладкая функция.



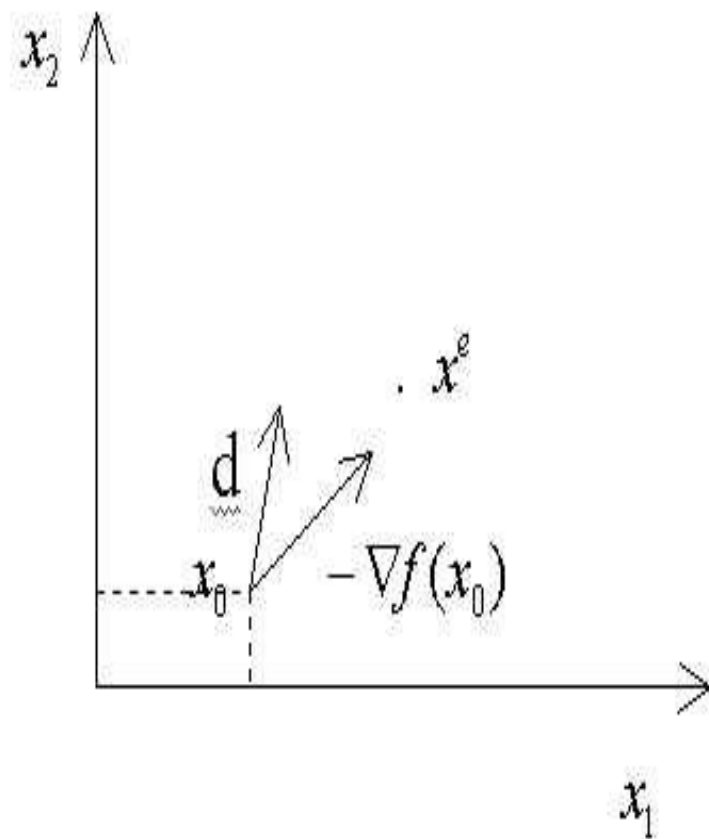
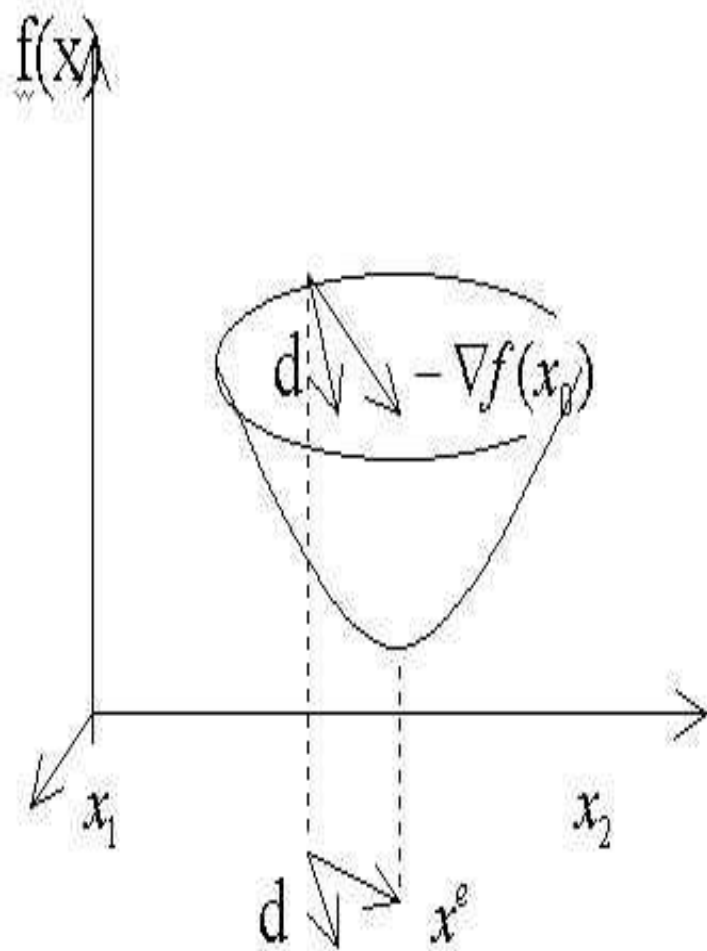
$$\operatorname{grad} f(x_0) = \nabla_x f(x_0) = \left\{ \frac{df}{dx_i} \right\}_{i=1,n}; \nabla_x f(x) = \left[\frac{df}{dx_1}, \frac{df}{dx_2}, \dots, \frac{df}{dx_n} \right]^T$$

для выпуклой функции:

$$f(x) \geq f(x_0) + \nabla_x^T f(x_0)(x - x_0)$$

фундаментальное неравенство выпуклых функций.

Дифференцирование функций



Дифференцирование функций.

Разложение функции в ряд Тейлора

В случае n -мерного пространства переменных - разложение в ряд Тейлора.

Пусть $f: X \rightarrow Y; X \subset E^n, Y \subset E^1$, тогда, если $f(x)$ n раз дифференцируема в т. x_0 , то $f(x_0 + \Delta x) = f(x_0) + \nabla^T f(x_0)\Delta x + \frac{1}{2}\Delta x^T H(x_0)\Delta x + \dots + O(\|\Delta x\|^T)$; где O - остаточный член в форме Пеано;

$$\nabla^T f(x_0) = \text{grad} f(x_0) = \left\{ \frac{\partial f(x_0)}{\partial x_i} \right\}_{i=1,n} \quad - \text{градиент функции } f(x) \text{ в т. } x_0;$$
$$H(x_0) = \left\{ \frac{\partial^2 f(x_0)}{\partial x_i \partial x_j} \right\}_{\substack{i=1,n \\ j=1,n}} \quad - \text{матрица Гессе 2-х частных производных в т. } x_0.$$

Если $\det H(x_0) \geq 0$, функция $f(x)$ в т. x_0 выпукла (если $>$ – строго).

Если $\det H(x_0) \leq 0$, функция $f(x)$ в т. x_0 вогнута (если $<$ – строго).

Если $\det H(x_0) = 0$, функция $f(x)$ в т. x_0 имеет перегиб.

Дифференцирование функций.

Матрица Якоби. Якобиан.

Пусть g - операторное отображение: $g: X \rightarrow Z, X \subset E^n, Z \subset E^m$,

Тогда $grad\ g(x_0) = J(x_0)$ – матрица Якоби, где $J(x_0) = \left[\frac{\partial g_j(x_0)}{\partial x_i} \right]_{\substack{i=1,n \\ j=1,m}}$
а $\det(J(x_0))$ – Якобиан. $\det(J(x_0)) = |J(x_0)|$.

Если $\det J(x) \neq 0$, то система уравнений $gj(x) = C$ имеет решение.

$\forall x \in X$. В этом случае существует обратное отображение

$$g^{-1}(x_0) \rightarrow x_0, [m.e. g^{-1} : Z \rightarrow X].$$

Это имеет большое значение при решении задач с неявно заданными функциями.

Дифференцирование функций

Классы дифференцируемых функций.

Пусть $f: X \rightarrow Y; X \subset E^n, Y \subset E^1$, тогда

$f(x) \in C^0(X)$ – класс непрерывных функций;

$f(x) \in C^1(X)$ – класс 1 раз дифференцируемых функций;

$f(x) \in C^2(X)$ – класс 2 раза дифференцируемых функций;

.....

$f(x) \in C^n(X)$ – класс n раз дифференцируемых функций.

Необходимые и достаточные условия существования экстремума функции.

Условия существования экстремума без учета ограничений.

Необходимые и достаточные условия выпуклости функций.

Теорема1. (Необходимые условия).

Пусть $f: X \rightarrow Y; X \subset E^n, Y \subset E^1 \quad \|x - \bar{x}\| \leq \varepsilon$
 $y = f(x); \forall x, \bar{x} \in X, y \in Y, f(x) \in C^1(X)$, тогда для того, чтобы $f(x)$ была выпуклой в $U_\varepsilon(x)$, необходимо, чтобы

$$f(x) \geq f(\bar{x}) + \nabla_x^T f(\bar{x}) U_\varepsilon(\bar{x}) \quad , \quad \text{где } \|x - \bar{x}\| \leq \varepsilon, U_\varepsilon(\bar{x})$$

Теорема2.(достаточные условия).

Пусть $f: X \rightarrow Y; X \subset E^n, Y \subset E^1$ и $\exists \varepsilon$, для которой $\|x - \bar{x}\| \leq \varepsilon, U_\varepsilon(\bar{x})$,
 $\forall x, \bar{x} \in X, y \in Y, f(x) \in C^2(x)$, тогда для того, чтобы $f(x)$, была выпуклой в $U_\varepsilon(\bar{x})$ достаточно, чтобы $\det H(\bar{x}) > 0$.

Необходимые и достаточные условия существования экстремума функции.

Необходимые и достаточные условия существования экстремума функции без ограничений.

Теорема3. Пусть $f: X \rightarrow Y; X \subset E^n, Y \subset E^1; y = f(x); \forall x \in X, y \in Y, f(x) \in C^2(X)$.

Тогда, для того чтобы $x^e = \text{Argextr}_{x \in X} f(x)$ необходимо и достаточно, чтобы

были выполнены условия:
$$\begin{cases} \nabla f(x^e) = 0 \\ \det H(x^e) \neq 0 \end{cases}$$

При этом:
$$\begin{cases} \nabla f(x^e) = 0 \\ \det H(x^e) > 0 \end{cases} \quad \text{приводит к} \quad x^e = \text{Arg min}_{x \in X} f(x)$$

$$\begin{cases} \nabla f(x^e) = 0 \\ \det H(x^e) \leq 0 \end{cases} \quad \text{приводит к} \quad x^e = \text{Arg max}_{x \in X} f(x)$$

Методы оптимизации в машинном обучении

Основные лосс-функции в машинном обучении.

Для задач регрессии

1. Mean Squared Error (MSE, Среднеквадратичная ошибка)
2. Mean Absolute Error (MAE, Средняя абсолютная ошибка)

Для задач классификации

1. Binary Cross-Entropy (Бинарная кросс-энтропия)
2. Categorical Cross-Entropy (Категориальная кросс-энтропия)

Для задач ранжирования и других

1. KL-Divergence (Kullback-Leibler Divergence)
2. Contrastive Loss

Задача оптимизации в машинном обучении

Общая формулировка:

$$\min_{\mathbf{w}} f(\mathbf{w}) = L(\mathbf{w}) + \lambda R(\mathbf{w})$$

$\mathbf{w}$

где:

- $\mathbf{w} \in \mathbb{R}^n$ - вектор параметров модели (веса)
- $L(\mathbf{w})$ - функция потерь (loss function)
- $R(\mathbf{w})$ - регуляризатор
- λ - коэффициент регуляризации

Типичные задачи:

- Обучение нейронных сетей: $\mathbf{w}$ - веса и смещения всех слоев
- Линейная регрессия: $f(\mathbf{w}) = \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2 + \lambda \|\mathbf{w}\|^2$
- Логистическая регрессия: $f(\mathbf{w}) = -\sum \log \sigma(\mathbf{y}_i \mathbf{w}^T \mathbf{x}_i) + \lambda \|\mathbf{w}\|^2$
- SVM: специальная формулировка с использованием двойственности

Зачем нужна оптимизация?

Центральная роль оптимизации в МО

Почему это важно:

1. Обучение = Оптимизация

- Любое обучение модели сводится к минимизации функции потерь
- Качество модели напрямую зависит от качества оптимизации

2. Вычислительная эффективность

- Современные модели: миллионы/миллиарды параметров
- GPT-3: 175 млрд параметров
- Нужны эффективные алгоритмы оптимизации

3. Обобщающая способность

- Не просто минимум на обучающей выборке
- Важно избежать переобучения
- Регуляризация и правильный выбор оптимизатора

Вызовы:

- Невыпуклые функции потерь (нейронные сети)
- Высокая размерность
- Большие объемы данных (Big Data)
- Стохастичность

Градиентный спуск - основная идея

Принцип: Движение в направлении антиградиента

Алгоритм:

Инициализация: w_0

Для $t = 0, 1, 2, \dots$ $w_{t+1} = w_t - \eta \nabla f(w_t)$,

где:

- $\eta > 0$ - learning rate (скорость обучения)
- $\nabla f(w)$ - градиент функции потерь в точке w

Интуиция:

- Градиент указывает направление наибыстрейшего роста
- Антиградиент $\rightarrow$ направление наибыстрейшего убывания
- Делаем шаг в этом направлении

Теорема (для выпуклых функций):

При достаточно малом η и выпуклой $f(w)$: $\lim_{t \rightarrow \infty} f(w_t) = f(w^*)$,

где w^* - глобальный минимум

Batch Gradient Descent

Полный градиентный спуск

Для функции потерь на обучающей выборке:

$$f(w) = (1/n) \sum_{i=1}^n L(y_i, \hat{y}_i(w))$$

Градиент:

$$\nabla f(w) = (1/n) \sum_{i=1}^n \nabla L(y_i, \hat{y}_i(w))$$

Вычисляем градиент по **ВСЕЙ** выборке

Преимущества:

-  Точный градиент
-  Стабильная сходимость
-  Теоретические гарантии

Недостатки:

-  Медленно для больших данных ($n \rightarrow \infty$)
-  Нужно хранить все данные в памяти
-  Избыточные вычисления для похожих примеров

Стохастический градиентный спуск (SGD)

Stochastic Gradient Descent

Основная идея: Обновление параметров на каждом примере

Алгоритм:

```
python
```

```
for epoch in range(num_epochs):
```

Перемешиваем данные

```
shuffle(X_train, y_train)
```

```
for i in range(n):
```

Градиент на ОДНОМ примере

```
grad = compute_gradient(w, X_train[i], y_train[i])
```

```
w = w - learning_rate * grad
```

Математически:

$$w_{t+1} = w_t - \eta \nabla L(y_{i_t}, \hat{y}_{i_t}(w_t)),$$

где i_t - случайно выбранный индекс на шаге t

Ключевое свойство:

$$E[\nabla L(y_i, \hat{y}_i(w))] = \nabla f(w)$$

Градиент на случайном примере - несмещенная оценка полного градиента!

SGD vs Batch GD - сравнение

Mini-batch Gradient Descent - компромисс

Mini-batch SGD:

```
`python
for epoch in range(num_epochs):
    for batch in get_batches(X_train, y_train, batch_size):
        X_batch, y_batch = batch
        grad = compute_gradient(w, X_batch, y_batch)
        w = w - learning_rate * grad
```

Сравнение методов:

На практике:

- Batch size: 32, 64, 128, 256 (степени 2 для эффективности GPU)
- Mini-batch - стандарт в deep learning

Преимущества mini-batch:

- ☒ Эффективное использование GPU (параллелизм)
- ☒ Регуляризующий эффект шума
- ☒ Возможность онлайн-обучения

Метод	Размер батча	Скорость итерации	Шум градиента	Сходимость
-----	-----	-----	-----	-----
Batch GD	n (все)	Медленно	Нет	Гладкая
SGD	1	Быстро	Высокий	Зашумленная
Mini-batch	32-512	Средне	Средний	Баланс

Проблема обусловленности

Число обусловленности и скорость сходимости

Число обусловленности (для квадратичной функции):

$$\kappa = \lambda_{\max} / \lambda_{\min}$$

где $\lambda_{\max}$, $\lambda_{\min}$ - наибольшее и наименьшее собственные числа Гессиана $H = \nabla^2 f(w)$

Влияние на сходимость:

Для квадратичной функции $f(w) = \frac{1}{2}w^T H w - b^T w$:

Скорость сходимости $\propto (\kappa - 1)/(\kappa + 1)$

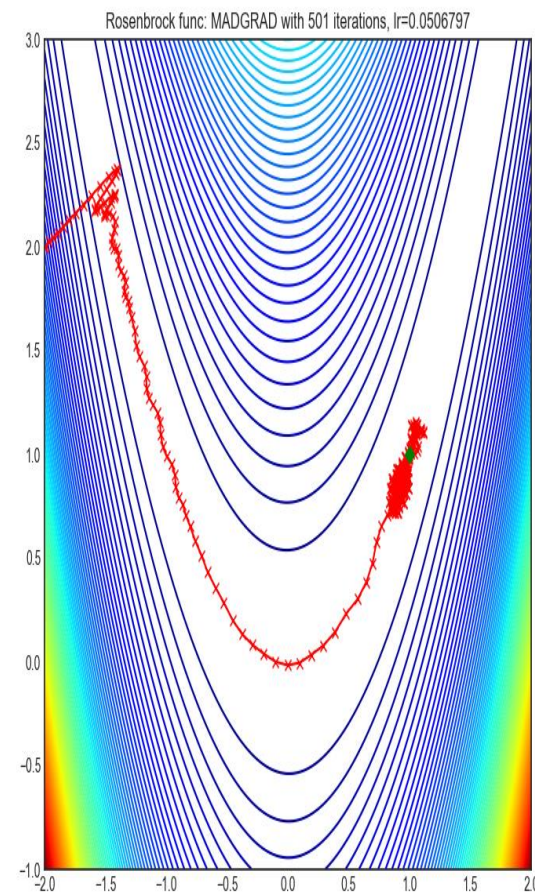
Плохая обусловленность ($\kappa \gg 1$):

- ✗ Медленная сходимость
- ✗ Чувствительность к learning rate
- ✗ "Зигзагообразная" траектория

Визуализация: Линии уровня вытянутые («Банановая» функция)

Функция Розенброка

$$f(x_1, x_2) = 100(x_1 - x_2^2)^2 + (1 - x_2)^2$$



Методы оптимизации в машинном обучении

Основные методы оптимизации

Градиентные методы первого порядка

1. **SGD (Stochastic Gradient Descent)** – базовый метод, обновляет параметры по градиенту на мини-батче:

$$\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t, x_i)$$

Проблемы: чувствительность к learning rate (η), медленная сходимость.

2. **SGD with Momentum** – добавляет инерцию для ускорения сходимости:

$$v_{t+1} = \gamma v_t + \eta \nabla L(\theta_t)$$

$$\theta_{t+1} = \theta_t - v_{t+1}$$

(γ – коэффициент затухания, обычно ~ 0.9).

3. **Nesterov Accelerated Gradient (NAG)** – улучшенный momentum, учитывающий будущий градиент.

Методы оптимизации в машинном обучении

Адаптивные методы (Adaptive Optimizers)

Учитывают историю градиентов для адаптивного изменения learning rate:

AdaGrad – адаптирует шаг для каждого параметра:

$$\theta_{t+1} = \theta_t - \eta G_t + \epsilon \nabla L(\theta_t)$$

(G_t – сумма квадратов прошлых градиентов).

Проблема: learning rate может слишком уменьшиться.

RMSProp – исправляет AdaGrad, используя экспоненциальное скользящее среднее:

$$G_t = \beta G_{t-1} + (1 - \beta)(\nabla L(\theta_t))^2$$

- **Adam (Adaptive Moment Estimation)** – комбинирует momentum и RMSProp:

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1 - \beta_1) \nabla L(\theta_t) \text{ (скользящее среднее градиента)} \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) (\nabla L(\theta_t))^2 \text{ (скользящее среднее квадратов)} \end{aligned}$$

$$\theta_{t+1} = \theta_t - \eta \cdot m_t v_t + \epsilon$$

2.3. Методы второго порядка

- Используют информацию о кривизне (матрица Гессе H), но сложны для больших моделей:
- **Метод Ньютона** – $\theta_{t+1} = \theta_t - H^{-1} \nabla L(\theta_t)$
- **L-BFGS** – квази-ньютоновский метод, подходит для небольших задач.

ГРАДИЕНТНЫЕ МЕТОДЫ В МАШИННОМ ОБУЧЕНИИ И ИСКУССТВЕННЫХ НЕЙРОННЫХ СЕТЯХ

Batch gradient descent.

$$\theta = \theta - \eta \nabla_{\theta} J(\theta)$$

*Градиентный метод, который рассчитан на вычисление градиента для **всего набора данных (выборке - batch)**.*

Применение этого метода обладает следующими особенностями:

- 1) Требуется достаточно большого объема памяти.
- 2) Обладает низкой скоростью сходимости.
- 3) Низкой точностью.

ГРАДИЕНТНЫЕ МЕТОДЫ В МАШИННОМ ОБУЧЕНИИ И ИСКУССТВЕННЫХ НЕЙРОННЫХ СЕТЯХ

Stochastic gradient descent (SGD)

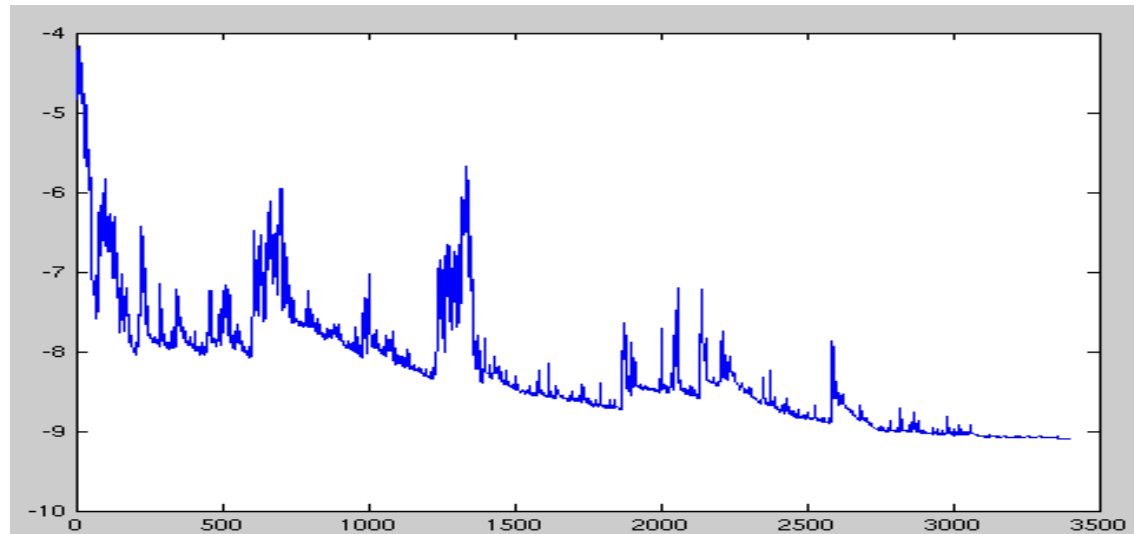
$$\theta = \theta - \eta \nabla_{\theta} J(\theta; x^{(i)}; y^{(i)})$$

В отличие от традиционного градиентного метода, метод стохастического градиента позволяет обрабатывать за одну итерацию относительно небольшой набор обучающих данных из всего набора данных (выборки - batch).

Применение этого метода позволяет избежать недостатки традиционного градиентного метода:

- 1) Требуется значительно меньше необходимого объема памяти.
- 2) Повышается скорость сходимости.
- 3) Увеличивается робастность к невыпуклости целевых функций.
- 4) Возможна оптимизация многоэкстремальных целевых функций.

ГРАДИЕНТНЫЕ МЕТОДЫ В МАШИННОМ ОБУЧЕНИИ И ИСКУССТВЕННЫХ НЕЙРОННЫХ СЕТЯХ



Флуктуации целевой функции в процессе работы алгоритма

ГРАДИЕНТНЫЕ МЕТОДЫ В МАШИННОМ ОБУЧЕНИИ И ИСКУССТВЕННЫХ НЕЙРОННЫХ СЕТЯХ

Mini-batch gradient descent

$$\theta = \theta - \eta \nabla_{\theta} J(\theta; x^{(i:i+n)}; y^{(i:i+n)})$$

*За каждую эпоху отбирается n данных ($n = 50 - 250$) небольшой набор обучающих данных из всего набора данных (выборки - *batch*).*

Применение этого метода дает следующие преимущества:

- 1) Уменьшает дисперсию разброса параметров что ведет к более устойчивой сходимости.
- 2) Позволяет использовать матричные и тензорные операции, входящие в состав современных библиотек методов глубокого обучения (смотри слайд 28).

Все это делает использование SGD на минибатчах очень эффективными для решения задач по глубокому обучению.

Далее мы рассмотрим дальнейшее развитие методов SGD.

ГРАДИЕНТНЫЕ МЕТОДЫ В МАШИННОМ ОБУЧЕНИИ И ИСКУССТВЕННЫХ НЕЙРОННЫХ СЕТЯХ

Momentum

$$v_t = v_{t-1} + \eta \nabla_{\theta} J(\theta)$$
$$\theta = \theta - v_t$$

Этот метод, применительно к SGD позволяет преодолевать овраги. Кроме того, он позволяет увеличить скорость сходимости алгоритмов SGD.

Метод моментов был предложен в 1964 году Б.Т. Поляком.

Другое название этого метода - «метод тяжелого шарика». Он рассматривался нами раньше в лекции посвященной многоэкстремальной оптимизации.

В 1986 году этот метод был использован Джеффри Хинтоном и Дэвидом Руммельхартом для обучения нейронных сетей методом обратного распространения ошибки.

ГРАДИЕНТНЫЕ МЕТОДЫ В ИСКУССТВЕННЫХ НЕЙРОННЫХ СЕТЯХ

Адаптивные методы

- 1. NAG. (Nesterov accelerated gradient)**
- 2. Adagrad. (Adapted gradient)**
- 3. Adadelat. (Extension of Adagrad)**
- 4. RMSProp. (Adaptive Root Mean Squared)**
- 5. Adam. (Adaptive moment estimation)**
- 6. AdaMax. (Modification Adam)**
- 7. Nadam. (Nesterov modification Adam)**

ГРАДИЕНТНЫЕ МЕТОДЫ В ИСКУССТВЕННЫХ НЕЙРОННЫХ СЕТЯХ

NAG. (Nesterov accelerated gradient)

$$v_t = v_{t-1} + \eta \nabla_{\theta} J(\theta - \gamma v_{t-1})$$

$$\theta = \theta - v_t$$

ГРАДИЕНТНЫЕ МЕТОДЫ В МАШИННОМ ОБУЧЕНИИ И ИСКУССТВЕННЫХ НЕЙРОННЫХ СЕТЯХ

Adagrad. (Adapted gradient)

$$\begin{aligned}g_{t,i} &= \nabla_{\theta_t} J(\theta_{t,i}) \\ \eta \\ \theta_{t+1,i} &= \theta_{t,i} - \frac{\eta}{\sqrt{G_{t,ii} + \varepsilon}} g_{t,i} \\ G_{t,ii} &= G_{t-1,ii} + g_{t,i}^2\end{aligned}$$

ГРАДИЕНТНЫЕ МЕТОДЫ В МАШИННОМ ОБУЧЕНИИ И ИСКУССТВЕННЫХ НЕЙРОННЫХ СЕТЯХ

Adadelata. (Extension of Adagrad)

$$\theta_{t+1} = \theta_t + \Delta\theta_t.$$
$$\Delta\theta_t = - \frac{RMS[\Delta\theta_{t-1}]}{RMS[g_t]} g_t.$$

$$M[\Delta\theta_t^2] = \gamma M[\Delta\theta_{t-1}^2] + (1 - \gamma) \Delta\theta_t^2.$$

$$M[g_t^2] = \gamma M[g_{t-1}^2] + (1 - \gamma) g_t^2.$$

$$RMS[\Delta\theta_t] = \sqrt{M[\Delta\theta_t^2] + \varepsilon}.$$

$$RMS[g_t] = \sqrt{M[g_t^2] + \varepsilon}$$

.

ГРАДИЕНТНЫЕ МЕТОДЫ В МАШИННОМ ОБУЧЕНИИ И ИСКУССТВЕННЫХ НЕЙРОННЫХ СЕТЯХ

RMSProp. (Adaptive Root Mean Squared)

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{M[g_t^2] + \varepsilon}} g_t.$$

$$M[g_t^2] = 0.9M[g_{t-1}^2] + 0.1 g_t^2.$$

ГРАДИЕНТНЫЕ МЕТОДЫ В МАШИННОМ ОБУЧЕНИИ И ИСКУССТВЕННЫХ НЕЙРОННЫХ СЕТЯХ

Adam. (Adaptive moment estimation)

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t + \varepsilon}} \hat{m}_t \quad .$$

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t.$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2.$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}.$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}.$$

$$\beta_1 = \beta_2 = 0.98.$$

ГРАДИЕНТНЫЕ МЕТОДЫ В МАШИННОМ ОБУЧЕНИИ И ИСКУССТВЕННЫХ НЕЙРОННЫХ СЕТЯХ

AdaMax. (Modification Adam)

$$\theta_{t+1} = \theta_t - \frac{\eta}{u_t} \hat{m}_t.$$

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t.$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2.$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}.$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}.$$

$$u_t = \max(\beta_2 v_{t-1}, |g_t|)$$

$$\eta = 0.002; \beta_1 = 0.9; \beta_2 = 0.999.$$

ГРАДИЕНТНЫЕ МЕТОДЫ В МАШИННОМ ОБУЧЕНИИ И ИСКУССТВЕННЫХ НЕЙРОННЫХ СЕТЯХ

Nadam. (Nesterov modification Adam)

Является комбинацией NAG и Adam

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t + \varepsilon}} \left(\beta_1 \widehat{m}_t + \frac{(1 - \beta_1) g_t}{1 - \beta_1^t} \right).$$

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t.$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2.$$

$$\widehat{m}_t = \frac{m_t}{1 - \beta_1^t}.$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}.$$

$$\eta = 0.002; \beta_1 = 0.9; \beta_2 = 0.999.$$

МЕТОД L-BFGS-B

(Limited-memory BFGS with Bounds)

- Квази-ньютоновский метод с ограниченной памятью
- Модификация BFGS для решения двух проблем:
- Высокое потребление памяти
- Поддержка граничных ограничений
- Limited Memory (L): Вместо хранения полной матрицы B_k , хранятся только m последних пар векторов $\{s_i, y_i\}$ (обычно $m \approx 10$).
- Произведение $B_k \nabla f$ вычисляется на лету с помощью двухуровневой рекурсии.
- Поддержка ограничений вида: $l_i \leq x_i \leq u_i$ (box constraints)

Алгоритм

Используется метод проекции градиента для определения активного множества (переменных на границах)

Вычисляется точка Коши (Cauchy point) — движение по антиградиенту с учетом границ

В подпространстве свободных переменных решается задача минимизации квадратичной модели с L-BFGS аппроксимацией

Линейный поиск с соблюдением границ

L-BFGS - Limited Memory BFGS

- **Проблема BFGS:** $O(n^2)$ память недопустимо для больших n
- **Идея L-BFGS:**
- Не хранить матрицу B_t явно
- Хранить только m последних пар векторов $\{s_i, y_i\}$
- Вычислять $B_t \nabla f(w_t)$ неявно
- **Алгоритм L-BFGS:**
- Память: $O(mn)$, обычно $m = 5-20$
- Время: $O(mn)$ на итерацию

Сравнение методов и практические рекомендации

-----	-----	-----	-----	-----	-----
	-				
SGD	O(n)	Низкие	η	Медленная	Простые задачи
Momentum	O(n)	Низкие	η, γ	Средняя	Общее
NAG	O(n)	Низкие	η, γ	Средняя+	Выпуклая оптимизация
AdaGrad	O(n)	Средние	η	Может застрять	Разреженные данные
RMSProp	O(n)	Средние	η, β	Хорошая	RNN
Adam	O(n)	Средние	η, β_1, β_2	Хорошая	Стандарт DL
L-BFGS	O(mn)	Высокие	m	Быстрая	Малые/средние сети

Практические рекомендации:

- По умолчанию:
- 🏆 Adam (lr=0.001) - для большинства задач
- 🥈 SGD + Momentum (lr=0.01-0.1, momentum=0.9) - для хорошей генерализации

Специальные случаи:

- CV (ResNet, etc.): SGD + Momentum + LR schedule
- NLP (Transformers): Adam/AdamW
- RNN: RMSProp или Adam
- Мало данных: L-BFGS
- **Советы:**
- Начать с Adam для быстрого прототипирования
- Для production: попробовать SGD + Momentum с тщательным подбором LR

ЛИТЕРАТУРА

1. Васильев Ф.П. "Методы оптимизации"

Издательство: Факториал Пресс, 2002 (переиздания)

Классический фундаментальный учебник

Охватывает линейное и нелинейное программирование, численные методы

2. Пантелеев А.В., Летова Т.А. "Методы оптимизации в примерах и задачах"

Издательство: Высшая школа, 2005 (переиздания 2015)

Практико-ориентированный подход

Содержит большое количество решенных задач

3. Карманов В.Г. "Математическое программирование"

Издательство: Физматлит, 2004 (переиздания)

Теория и численные методы

Подходит для углубленного изучения

4. Аттетков А.В., Галкин С.В., Зарубин В.С. "Методы оптимизации"

Издательство: МГТУ им. Баумана, 2003 (переиздания 2014)

Учебник для технических вузов

Акцент на инженерных приложениях

5. Исмаилов А.Ф., Солодов М.В. "Численные методы оптимизации"

Издательство: Физматлит, 2005, 2008

Современные алгоритмы оптимизации

Строгое математическое изложение

1. Boyd S., Vandenberghe L. "Convex Optimization"

Cambridge University Press, 2004Классика! Наиболее цитируемая книга по выпуклой оптимизации. Доступна бесплатно на сайте авторов Теория + практические алгоритмы

2. Nocedal J., Wright S.J. "Numerical Optimization"

Springer, 2006 (2nd edition) Фундаментальный учебник по численным методам.

Охватывает линейное, нелинейное программирование, методы внутренней точки. Стандарт для курсов по оптимизации

3. Bertsekas D.P. "Convex Optimization Theory"

Athena Scientific, 2009 Строгое математическое изложение. Глубокое понимание теории выпуклости. Дополняет книгу Boyd & Vandenberghe

4. Beck A. "First-Order Methods in Optimization"

SIAM, 2017Современные методы первого порядка. Акцент на методах для больших данных и MLGradient descent, proximal methods, ADMM

5. Nesterov Y. "Lectures on Convex Optimization"

Springer, 2018 (2nd edition)От создателя accelerated gradient method.

Современные алгоритмы с оптимальной сложностьюПродвинутый уровень
Дополнительно (специализированные темы):

6. Bubeck S. "Convex Optimization: Algorithms and Complexity"

Foundations and Trends in Machine Learning, 2015. Краткий обзор (100 страниц). Доступен бесплатно онлайн. Связь оптимизации и машинного обучения

7. Wright S.J., Recht B. "Optimization for Data Analysis"

Cambridge University Press, 2022Самая современная! Оптимизация для анализа данных и MLStochastic methods, distributed optimization

Онлайн-ресурсы: Курсы Stanford (Boyd), MIT (Bertsekas) — доступны бесплатно

Arxiv.org — последние исследования в области оптимизации

Спасибо за внимание!

Каганов Юрий Тихонович

yurijkaganov@gmail.com

kaganov_y.t@bmstu.ru